Information Retrieval

Prof. Alberto Sillitti

6. Index construction

INNOPOLIS UNIVERSITY

Index construction

- How do we construct an index?
- What strategies can we use with limited main memory?

Recall index construction

- Documents are parsed to extract words and these are saved with the Document ID.

Doc 1

I did enact Julius Caesar I was killed i' the Capitol; Brutus killed me.

Doc 2

So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious



Term	Doc #
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Recall index construction

- After all documents have been parsed, the inverted file is sorted by terms.

We focus on this sort step.

Term	Doc #
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2



Term	Doc #
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

Collection Reuters Corpus Volume 1 (RCV1)

- One year of Reuters newswire (part of 1995 and 1996)

Symbol	Statistic	Value
N	documents	800,000
L	avg. tokens per doc	200
M	terms	400,000
	avg. bytes per token (incl. spaces/punctuation)	6
	avg. bytes per token (excl. spaces/punctuation)	4.5
	avg. Bytes per term	7.5
	non-positional postings	100,000,000

Sort-based index construction

- As we build the index, we parse docs one at a time.
  - The final postings for any term are incomplete until the end.
- At 8 bytes per *(termID, docID)*, demands a lot of space for large collections.
- T = 100,000,000 in the case of RCV1
  - So … we can do this in memory today, but typical collections are much larger. E.g., the *New York Times* provides an index of >150 years of newswire
- Thus: We need to store intermediate results on disk.

Scaling index construction

- In-memory index construction does not scale
  - Can’t stuff entire collection into memory, sort, then write back
- How can we construct an index for very large collections?
- Taking into account hardware constraints. . .
  - Memory, disk, speed, etc.

Hardware basics

- Servers used in IR systems now typically have several GB of main memory, sometimes tens of GB.
- Available disk space is several (2–3) orders of magnitude larger.
- Fault tolerance is very expensive: It’s much cheaper to use many regular machines rather than one fault tolerant machine.

Hardware basics

- Access to data in memory is much faster than access to data on disk.
- In traditional disks:
  - Disk seeks: No data is transferred from disk while the disk head is being positioned.
  - Therefore: Transferring one large chunk of data from disk to memory is faster than transferring many small chunks.
  - Disk I/O is block-based: Reading and writing of entire blocks (as opposed to smaller chunks) (8KB to 256 KB)

Hardware basics

- In SSD disks:
  - Seek time is almost 0
  - The disk is still organized in blocks

Sort using disk as “memory”?

- Can we use the same index construction algorithm for larger collections, but by using disk instead of memory?
- No: Sorting T = 100,000,000 records on disk is too slow (even on SSD)
- We need an *external* sorting algorithm.

BSBI: Blocked Sort-Based Indexing

- 8-byte records (termID, docID)
- These are generated as we parse docs
- Must now sort 100M such 8-byte records by termID
- Define a Block ~ 10M such records
  - Can easily fit a couple into memory
  - Will have 10 such blocks to start with
- Basic idea of algorithm:
  - Accumulate postings for each block, sort, write to disk
  - Then merge the blocks into one long sorted order

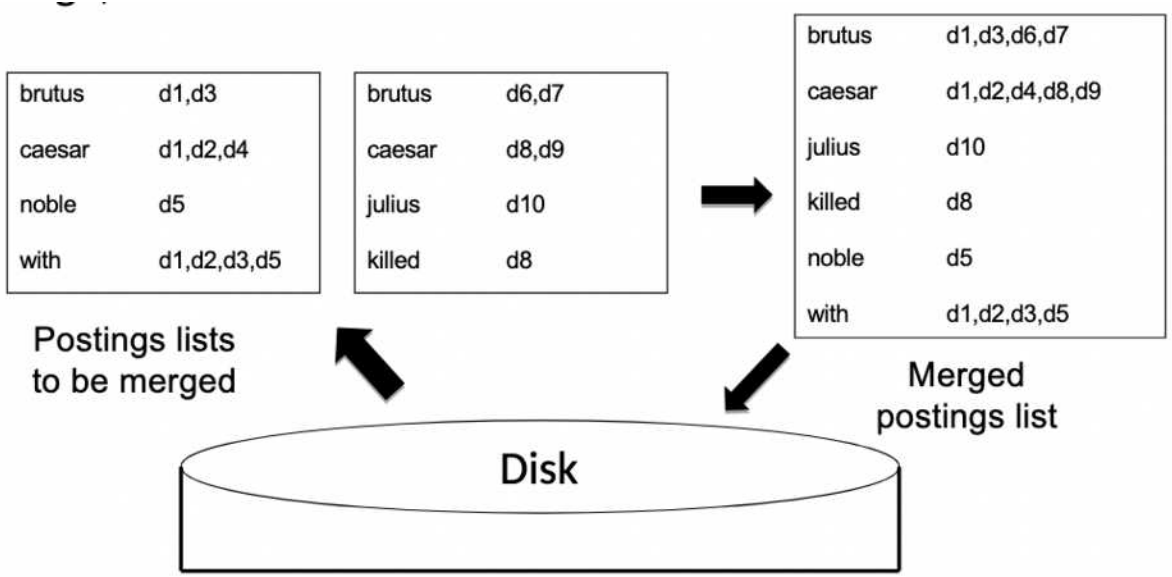
Sorting 10 blocks of 10M records

- First, read each block and sort within:
  - Quicksort takes O($N \ln N$) expected steps
  - In our case $N=10M$

- 10 times this estimate – gives us 10 sorted *runs* of 10M records each.
- Done straightforwardly, need 2 copies of data on disk
  - But can optimize this

How to merge the sorted runs?

- Can do binary merges, with a merge tree of $\log_2 10$ = 4 layers.
- During each layer, read into memory runs in blocks of 10M, merge, write back.



How to merge the sorted runs?

- But it is more efficient to do a multi-way merge, where you are reading from all blocks simultaneously
  - Open all block files simultaneously and maintain a read buffer for each one and a write buffer for the output file
  - In each iteration, pick the lowest termID that hasn’t been processed using a priority queue
  - Merge all postings lists for that termID and write it out

Remaining problem with sort-based algorithm

- Our assumption was: we can keep the dictionary in memory.
- We need the dictionary (which grows dynamically) to implement a term to termID mapping.

SPIMI: Single-Pass In-Memory Indexing

- **Key idea 1:** Generate separate dictionaries for each block – no need to maintain term-termID mapping across blocks.
- **Key idea 2:** Don’t sort. Accumulate postings in postings lists as they occur.
- With these two ideas we can generate a complete inverted index for each block.
- These separate indexes can then be merged into one big index.

Distributed indexing

- For web-scale indexing:
  - must use a distributed computing cluster
- Individual machines are fault-prone
  - Can unpredictably slow down or fail
- How do we exploit such a pool of machines?

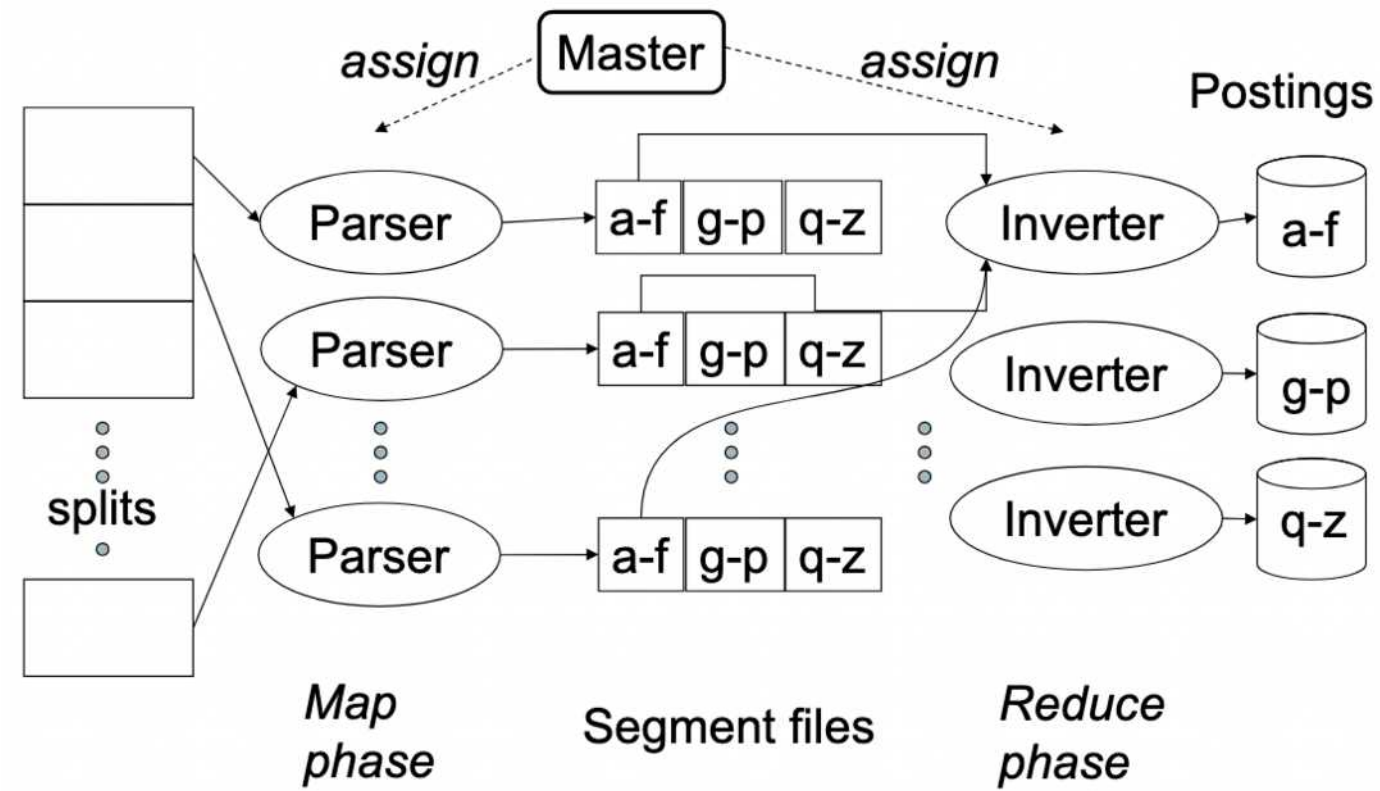
Distributed indexing

- Maintain a *master* machine directing the indexing job.
- Break up indexing into sets of (parallel) tasks.
- Master machine assigns each task to an idle machine from a pool.

Parallel tasks

- We will use two sets of parallel tasks
  - Parsers
  - Inverters
- Break the input document collection into *splits*
- Each split is a subset of documents (corresponding to blocks in BSBI/SPIMI)

Data flow


assign
Master
assign
Postings
Parser
Parser
Parser
a-f
g-p
q-z
a-f
g-p
q-z
a-f
g-p
q-z
Inverter
Inverter
Inverter
a-f
g-p
q-z
splits
Map phase
Segment files
Reduce phase

Parsers

- Master assigns a split to an idle parser machine
- Parser reads a document at a time and emits (term, doc) pairs
- Parser writes pairs into *j* partitions
- Example: Each partition is for a range of terms' first letters
  - (e.g., ***a-f***, ***g-p***, ***q-z***) – here *j* = 3.
- Now to complete the index inversion

Inverters

- An inverter collects all (term,doc) pairs (= postings) for one term-partition.
- Sorts and writes to postings lists

Index construction

- Index construction was just one phase.
- Another phase: transforming a term-partitioned index into a document-partitioned index.
  - *Term-partitioned*: one machine handles a subrange of terms
  - *Document-partitioned*: one machine handles a subrange of documents
- Most search engines use a document-partitioned index … better load balancing, etc.

Dynamic indexing

- Up to now, we have assumed that collections are static.
- They rarely are:
  - Documents come in over time and need to be inserted.
  - Documents are deleted and modified.
- This means that the dictionary and postings lists have to be modified:
  - Postings updates for terms already in dictionary
  - New terms added to dictionary

Simplest approach

- Maintain “big” main index
- New docs go into “small” auxiliary index
- Search across both, merge results
- Deletions
  - Invalidation bit-vector for deleted docs
  - Filter docs output on a search result by this invalidation bit-vector
- Periodically, re-index into one main index

Issues with main and auxiliary indexes

- Problem of frequent merges – you touch stuff a lot
- Poor performance during merge
- Actually:
  - Merging of the auxiliary index into the main index is efficient if we keep a separate file for each postings list.
  - Merge is the same as a simple append.
  - But then we would need a lot of files – inefficient for OS.
- Assumption for the rest of the lecture: The index is one big file.
- In reality: Use a scheme somewhere in between (e.g., split very large postings lists, collect postings lists of length 1 in one file etc.)

Further issues with multiple indexes

- Collection-wide statistics are hard to maintain
- E.g., when we speak of spell-correction: which of several corrected alternatives do we present to the user?
  - We may want to pick the one with the most hits
  - How do we maintain the top ones with multiple indexes and invalidation bit vectors?
  - One possibility: ignore everything but the main index for such ordering
- Will see more such statistics used in results ranking

Dynamic indexing at search engines

- All the large search engines now do dynamic indexing
- Their indices have frequent incremental changes
  - News items, blogs, new topical web pages
- But (sometimes/typically) they also periodically reconstruct the index from scratch
  - Query processing is then switched to the new index, and the old index is deleted